Competitive Programming

Combinatorial Games

Game Playing Basics

- A *combinatorial game*
  - Players have perfect knowledge of the game state.
  - Problem might ask who has a guaranteed win from a given state …
  - … assuming both players play perfectly.

Chocolate Division

Alf and Beata were two young adults living together a long, long time ago, before you could spend all your afternoons competing in programming. Their lives were thus much more boring than those of today's young adults. How could you even survive back then, you might ask yourself. The answer is simple: you make your own chocolate bars! Our two cohabitants loved making chocolate bars, and often had huge piles of them each afternoon. To avoid filling their entire living room with chocolate bars, Beata challenged her friend to a game every evening to eat up all the chocolate – the Chocolate Division game.

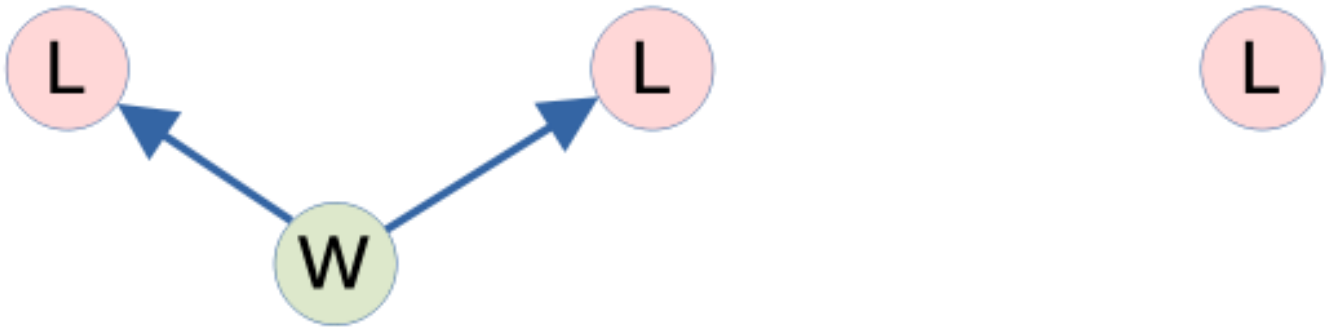
The Chocolate Division game is played by two players (in our case, Alf and Beata) using a chocolate bar consisting of R rows, each containing C squares of chocolate. A move consists of splitting a rectangular piece of chocolate either horizontally or vertically into two pieces, such that the two new pieces again have integer height and width. The first move is performed on the original chocolate bar. Each subsequent move can performed on any piece of chocolate that has been split off so far, as long as it doesn't have dimensions 1×1 (no split can be done on such a piece). Alf starts with the first move. If, at any point, a person can not perform their move because all remaining pieces have size 1×1 squares, that person loses.

Can you compute who wins the game, if both players play optimally?

Game Playing Basics

- A *combinatorial game*
  - Players have perfect knowledge of the game state.
  - Problem might ask who has a guaranteed win from a given state …
  - … assuming both players play perfectly.
- Typical solution techniques
  - Try / think about the game to find some kind of structure in the winning or losing solutions.
    - Often, there might be some kind of even / odd distinction
  - Write DP or Memoizing solution that can determine the win / lose status of any game state
  - Try to implement a guaranteed winning strategy and simulate the game.

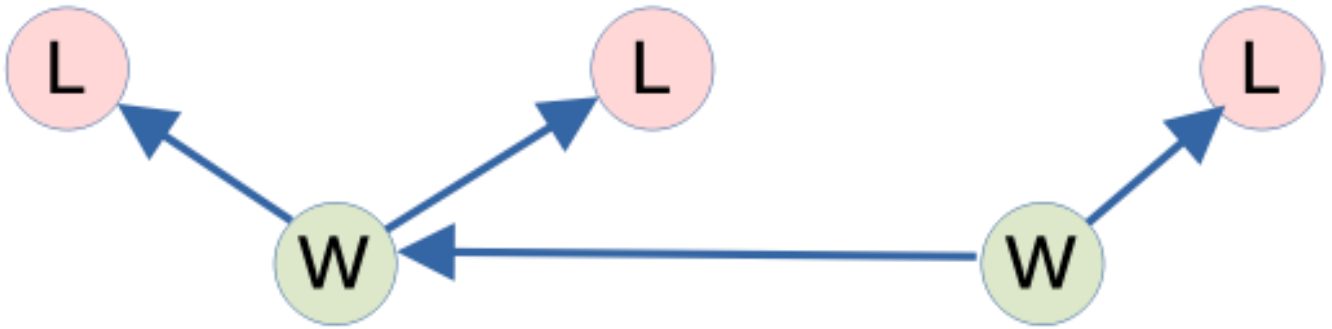
Winning and Losing States



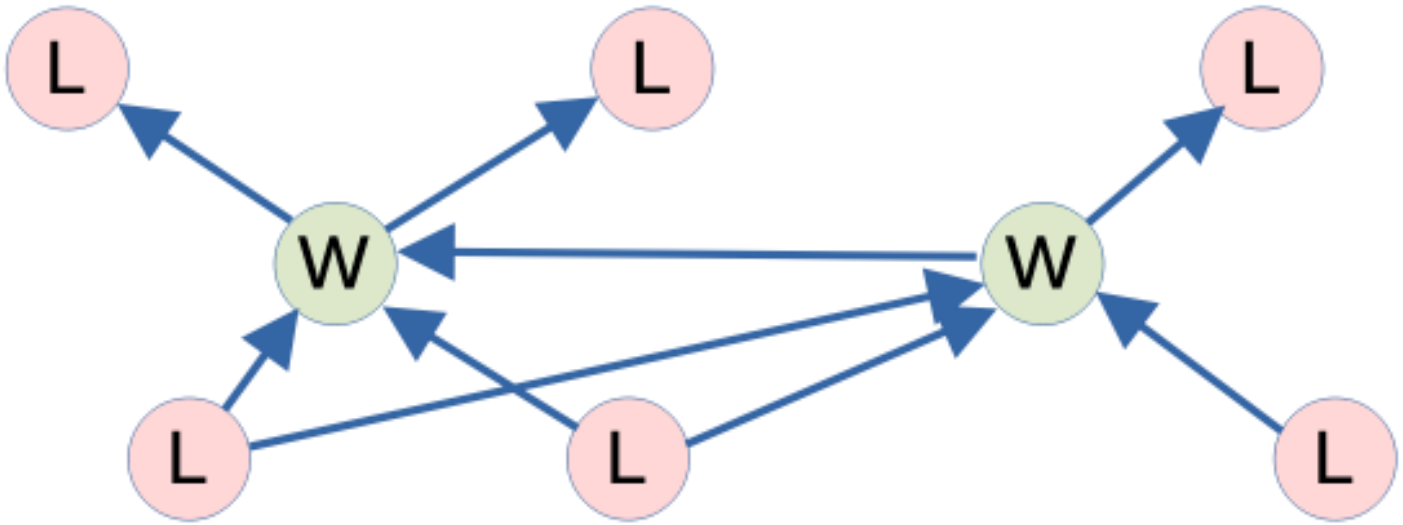
Winning and Losing States



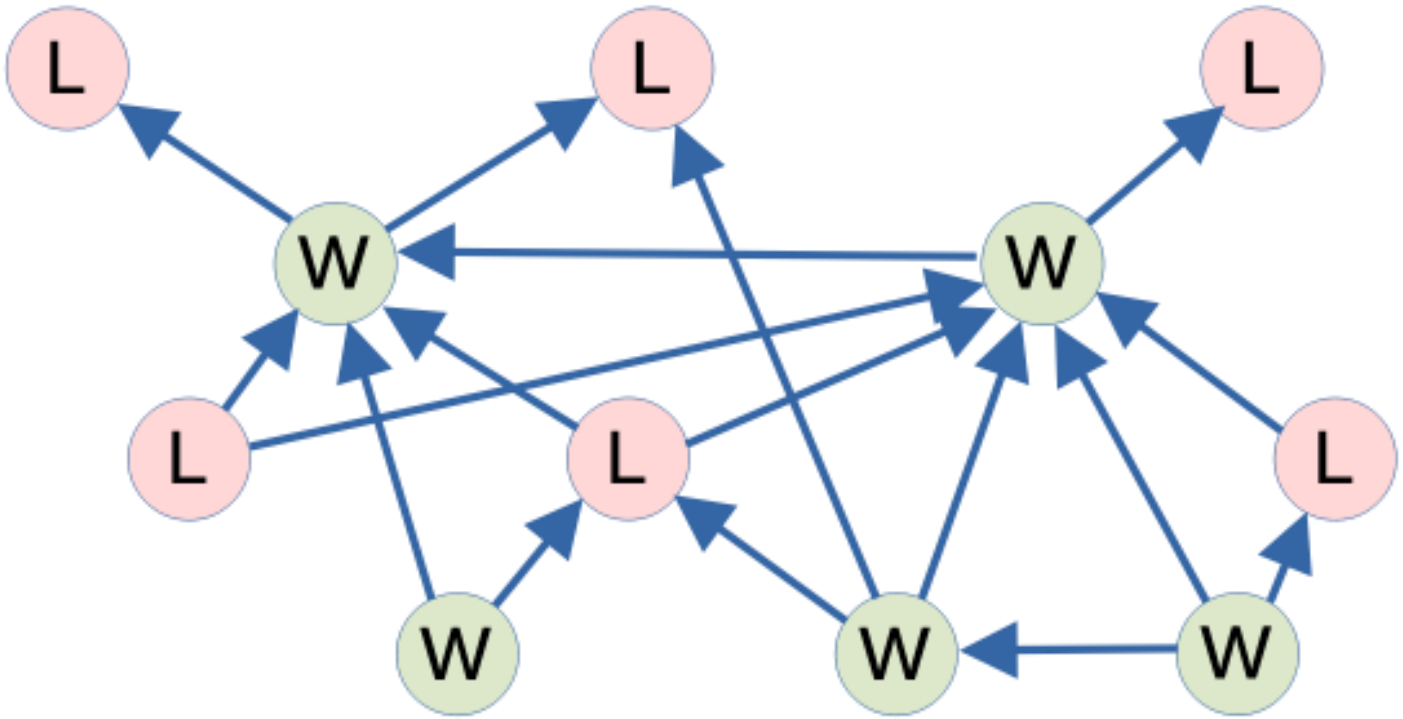
Winning and Losing States



Winning and Losing States



Winning and Losing States



Telling winning from losing

- For a winning state, there's at least one move that takes you to a losing state (for your opponent).
- For a losing state, all moves take you to a winning state (for your opponent)
- You can compute this, if the game doesn't have too many states.
  - Tic-Tack-Toe (fine)
  - Connect 4 (probably not)
- Sometimes, you can fold sets of states into equivalence classes